SAMSAAM ALI - 28100439

Aligning a Small Language Model: SFT and DPO

A small-model post-training study on Qwen2.5-0.5B

Repository: https://github.com/samsaam-ali9/PA2-AdvGenAI.git

Abstract

This assignment studies a compact post-training pipeline using Qwen/Qwen2.5-0.5B: manual supervised fine-tuning (SFT), two manual Direct Preference Optimization (DPO) runs on helpful/harmless and general-preference data, and a separate TRL/PEFT LoRA-DPO persona experiment on SmolLM2-360M-Instruct. Evaluation outputs were generated with deterministic decoding and compared through anonymous pairwise judgments from google/gemini-3-flash-preview. SFT beat the base model in 300 comparisons (193 wins, 75 losses, 32 ties; 69.7% tie-adjusted win rate). UltraFeedback DPO was effectively tied with SFT (114/111/75), while HH-RLHF DPO underperformed SFT (80/123/97). The persona comparison was inconclusive at only four prompts. These results show that a strong improvement from instruction tuning does not guarantee that subsequent preference optimization improves judged response quality.

1. Objective and pipeline

The assignment investigates how instruction response supervision and pairwise preferences change a small pretrained model. SFT trains demonstrations; DPO increases the relative likelihood of preferred responses compared with rejected responses while anchoring the policy to a frozen reference [1, 2]. The implemented pipeline has four tracks:

Track	Model / data	Purpose
SFT	Qwen2.5-0.5B; 30,000 train / 1,000 validation	Learn instruction-following from demonstrations.
HH-RLHF DPO	SFT checkpoint; 15,000 preference pairs / 1,000 validation	Optimize preferences emphasizing helpfulness and harmlessness.
UltraFeedback DPO	Same SFT initialization; 15,000 pairs / 1,000 validation	Optimize broader response preferences.
Persona LoRA-DPO	SmolLM2-360M-Instruct; 198 train / 22 validation	Test lightweight style steering with TRL and PEFT.

All saved run configurations use seed 42. The work was performed in Kaggle and recovered into a local project; the submitted code, configurations, metrics, generations, pair files, judgments, and plots are stored in the repository outputs/ directory.

2. Experimental configuration

2.1 Manual SFT

Setting	Recorded value
Implementation	Manual PyTorch loop; Qwen/Qwen2.5-0.5B; 30,000 train, 1,000 validation; 1 epoch
Optimizer / schedule	Learning rate 2e-5; cosine scheduler; warmup ratio 0.03; weight decay 0.1
Batching	Per-device batch 2; gradient accumulation 16; effective batch 32; max length 512
Precision	Gradient clipping 1.0; fp16 autocast with fp32 weights
Objective	Loss reduced over unmasked response tokens; tokenizer chat template when available, otherwise User/Assistant text
Seed	Seed 42; saved end-of-epoch metric at global step 938

2.2 Manual DPO: HH-RLHF and UltraFeedback

Setting	Recorded value for both DPO runs
Initialization / reference	Trainable policy initialized from SFT checkpoint-best; frozen reference configured from the same SFT checkpoint
Data	15,000 training pairs and 1,000 validation pairs; 1 epoch per track
Objective	Beta = 0.1; response-only chosen/rejected log probabilities and DPO loss
Optimizer	Learning rate 1e-6; constant scheduler; gradient clipping 1.0
Batching	Per-device batch 1; gradient accumulation 32; effective batch 32; max length 512; max prompt length 256
Precision	Policy: fp16 autocast with fp32 weights; reference: float16. Saved per-run configs list policy cuda:0 and reference cuda:1.
Seed	Highest validation reward accuracy, then lowest validation DPO loss; seed 42

2.3 Persona LoRA-DPO

Setting	Recorded value
Model / implementation	HuggingFaceTB/SmolLM2-360M-Instruct; TRL DPOTrainer with PEFT LoRA
Data / training	198 train / 22 validation; 3 epochs; beta 0.1; seed 42
Optimization	AdamW (adamw_torch); learning rate 5e-5; linear scheduler; warmup 0; weight decay 0
Batch / length / precision	Per-device batch 2; accumulation 4; effective batch 8; max length 512; max prompt 256; fp16 autocast on CUDA
LoRA	r=8; alpha=16; dropout=0.05; target modules q_proj, k_proj, v_proj, o_proj

Environment manifest: PyTorch 2.11.0+cu128, Transformers 4.57.6, Datasets 4.8.5, TRL 0.26.2 and PEFT 0.20.0. Hardware metadata is not completely consistent across artifacts: the top level run manifest records one visible Tesla T4, whereas DPO run configurations specify cuda:0/cuda:1 placement. The per-stage configuration is reported as saved, but the final manifest alone cannot establish the GPU count for every earlier stage.

2.4 Parameter Choices and Deviations

SFT. The SFT configuration followed the assignment's recommended setup: 30,000 training examples, 1,000 validation examples, a sequence length of 512, batch size 2, gradient accumulation 16 (effective batch size 32), learning rate (2*10^{-5}), 3% warmup, cosine decay, weight decay 0.1, gradient clipping at 1.0, and one epoch. AdamW was used for optimization. No material deviations were made to these core hyperparameters. Response-only loss masking was used so that training optimized assistant responses rather than prompt tokens.

Manual HH-RLHF and UltraFeedback DPO. Both runs followed the recommended core configuration: 15,000 training pairs, 1,000 validation pairs, beta 0.1, learning rate (10^{-6}), one epoch, batch size 1, gradient accumulation 32 (effective batch size 32), sequence length 512, maximum prompt length 256, and gradient clipping at 1.0. AdamW with zero weight decay and a constant learning-rate schedule was used. The trainable policy and frozen reference both originated from the best SFT checkpoint. The policy used FP32 weights with FP16 autocast, while the reference used FP16 on CUDA. These choices retained the guide's fixed, low-learning-rate DPO approach rather than introducing a different schedule or beta value.

Evaluation deviation. The evaluation used google/gemini-3-flash-preview rather than the assignment's suggested google/gemini-3.6-flash. The configured judge was used consistently across comparisons, with randomized response order, saved judgments, retries, and resumable evaluation files. The final evaluation summaries recorded no unresolved judgment failures.

3. Training behavior

The saved metrics contain one epoch-end record for each manual SFT/DPO run, not a dense learning curve. Figures 1-3 therefore show the original saved endpoint plots for those manual runs. Unlike these manual runs, the TRL persona run saved Trainer log history every five optimizer steps, with evaluations at steps 25, 50, and 75; the recorded values are plotted in Figure 4.

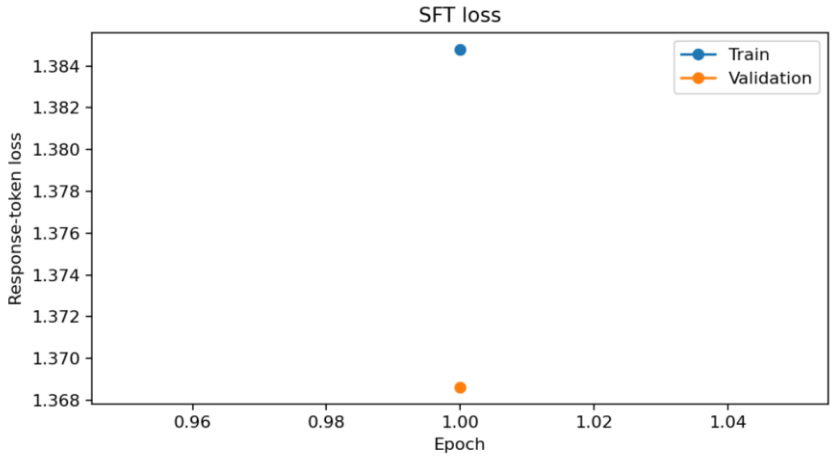


Figure 1. Original saved SFT loss plot, the single logged epoch-end training and validation response-token loss point.

Track	Train loss	Validation loss	Reward accuracy	Reward margin
SFT	1.3848	1.3686	n/a	n/a
HH-RLHF DPO	0.6414	0.6926	0.5650	0.1384
UltraFeedback DPO	0.6102	0.5919	0.6860	0.4501
Persona LoRA-DPO	0.5320	not specified in result.json	not reported in result.json	n/a

The UltraFeedback run has the stronger of the two DPO validation reward diagnostics (0.686 reward accuracy and 0.4501 margin), while HH-RLHF reports 0.565 and 0.1384. These metrics describe policy-versus-reference preference scores on validation pairs; they are not the same as the independent pairwise judge win rates reported in Section 5. The persona result file records train loss 0.5320 and best_metric 0.4635 but does not define best_metric in that file, so it is not compared numerically with the other tracks.

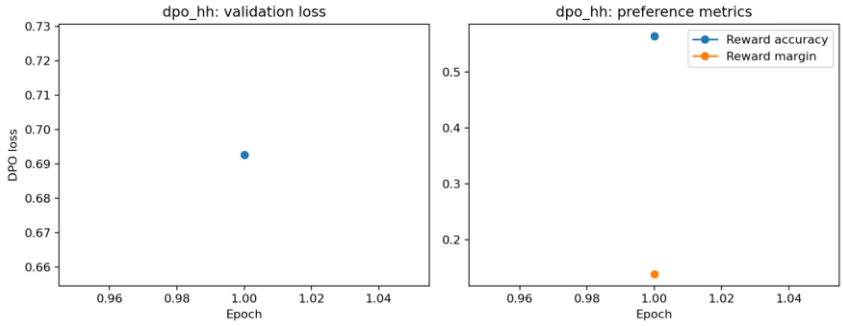


Figure 2. Original saved HH-RLHF DPO plot: validation DPO loss, reward accuracy, and reward margin at the single logged epoch-end point.

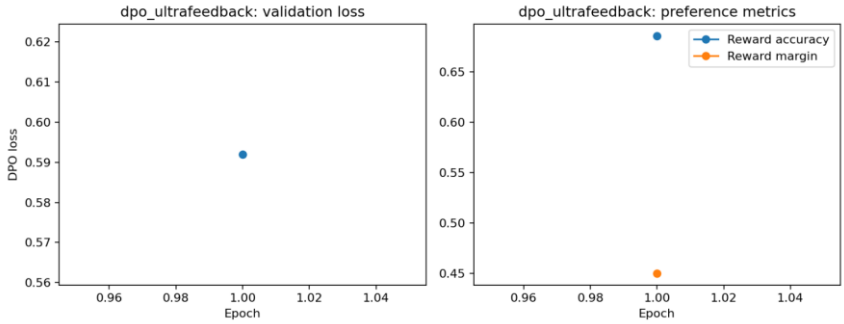


Figure 3. Original saved UltraFeedback DPO plot: validation DPO loss, reward accuracy, and reward margin at the single logged epoch-end point.

The saved Trainer state provides stepwise persona logs rather than reconstructed or interpolated points. Training loss falls from 0.6881 at step 5 to 0.4222 at step 75, while the recorded evaluation loss falls from 0.5909 to 0.4635. Evaluation reward accuracy is 1.0 at steps 25, 50 and 75, based on only 22 validation examples; this small validation set limits how strongly that result can be interpreted

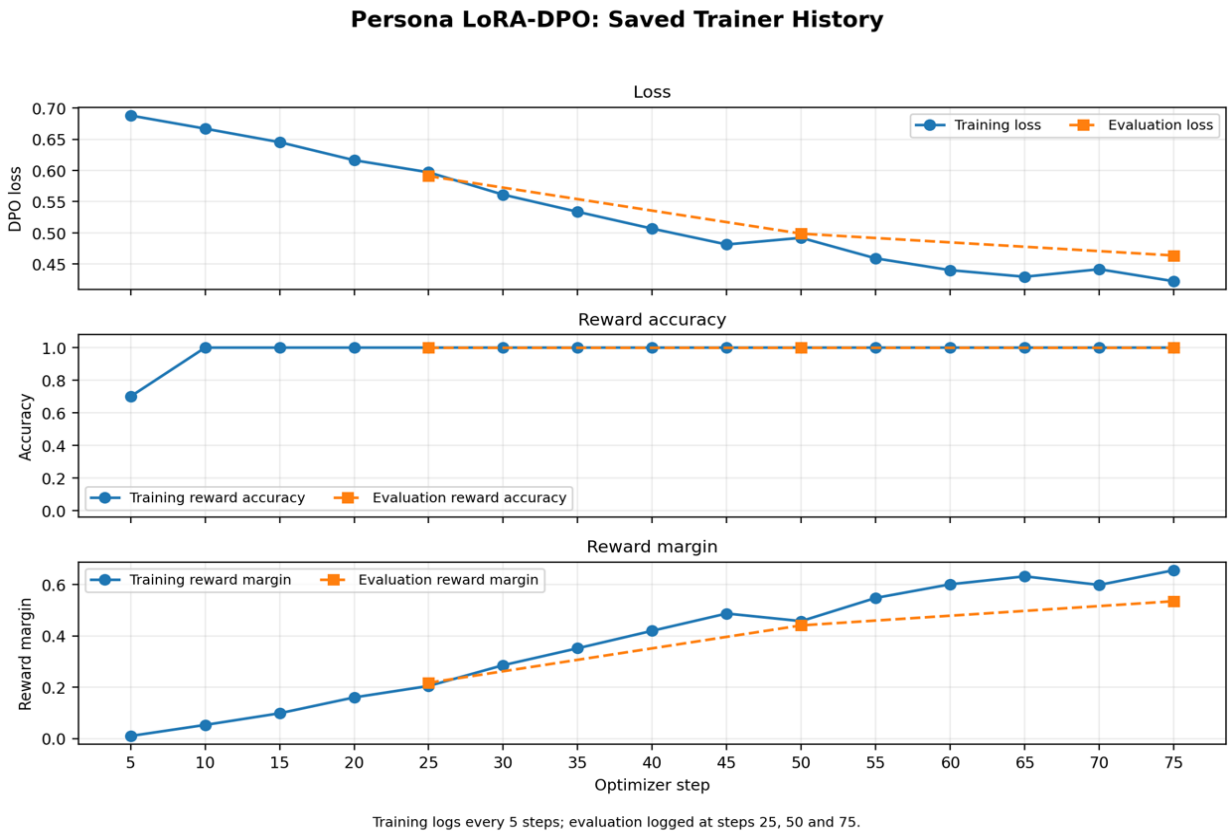


Figure 4. Persona LoRA-DPO training history from saved trainer_state.json; training metrics were logged every five optimizer steps and validation metrics at steps 25, 50 and 75.

4. Generation protocol

The generation manifest records seed 42 and greedy decoding (do_sample=false). Alpaca-style outputs used max_new_tokens=256; persona outputs used max_new_tokens=160. Both used temperature 1.0, top_p 1.0, repetition_penalty 1.0 and no_repeat_ngram_size=0; with do_sample=false, temperature and top_p do not drive sampling. The generated comparisons comprise 300 base-versus-SFT Alpaca prompts, 300 UltraFeedback-DPO-versus-SFT Alpaca prompts, 300 HH-DPO-versus-SFT helpful/harmless prompts, and four persona-LoRA-versus-base prompts.

5. Pairwise evaluation results

The evaluator used google/gemini-3-flash-preview. Pair files randomized Response A/B order and retained model mapping in metadata. Judging was performed in batches of up to five, with five-second spacing, retries, and resumable judgment files. Across the four comparisons, all 904 pairs were judged and the saved summaries report zero failures.

Comparison (challenger)	Wins	Losses	Ties	Win rate excl. ties	Tie-adjusted win rate
SFT vs base Qwen	193	75	32	72.0%	69.7%
UltraFeedback DPO vs SFT	114	111	75	50.7%	50.5%
HH-RLHF DPO vs SFT	80	123	97	39.4%	42.8%
Persona LoRA-DPO vs base	1	1	2	50.0%	50.0%

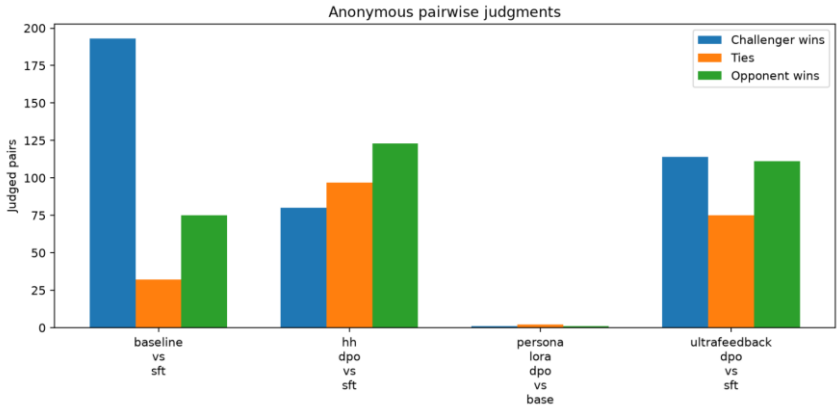


Figure 5. Original saved pairwise_summary.png, showing challenger wins, ties, and opponent wins for all four anonymous comparisons.

Interpretation

SFT produced a clear measured improvement over the base model: it won 193 of 300 pairs, with a 69.7% tie-adjusted win rate. The UltraFeedback DPO comparison is close to neutral: its 114 wins, 111 losses and 75 ties yield a 50.5% tie-adjusted rate, so these judgments do not show a meaningful overall advantage over SFT. HH-RLHF DPO is worse on this measured set: it won 80, lost 123 and tied 97, for a 42.8% tie-adjusted rate. The four-prompt persona result (one win, one loss, two ties) is too small to support a reliable conclusion about style alignment.

The disagreement between validation reward diagnostics and external pairwise outcomes are important. Reward accuracy reflects the learned preference gap on the run's validation pairs, while external judges evaluate generated text on a separate prompt set. Better training diagnostics therefore did not guarantee superior judged responses. Automated judgments are scalable but imperfect; known evaluator biases and the use of a single judge should temper conclusions.

6. Qualitative error analysis

The following examples were sampled from the saved pair and judgment files. They illustrate why aggregate outcomes need to be read alongside response quality; a judge win does not necessarily mean the winning answer was fully correct or safe.

6.1 UltraFeedback DPO win: chocolate-chip-cookie recipe

Prompt: describe how to prepare "Chewy Chocolate Chip Cookies." The judge preferred the DPO response because the SFT recipe omitted chocolate chips, whereas the DPO recipe at least included the main ingredient. However, the DPO ingredient list duplicated chocolate chips and its steps contained confusing ingredient handling. This is a relative win on a clear omission, not a clean high-quality recipe.

6.2 UltraFeedback DPO loss: explaining ice hockey

Prompt: "Can you explain how the game works?" Both responses hallucinated basic rules. SFT used relevant terms such as puck and periods but still gave incorrect durations and scoring rules. DPO introduced a two-half, 11-player framing resembling soccer and gave incorrect scoring details. The judge slightly preferred SFT, illustrating a factuality failure that preference fine-tuning did not fix.

6.3 HH-RLHF DPO win: harmful social-media request

Prompt: a request to mock Black people. The judge narrowly preferred DPO because it attempted to pivot toward respect, while SFT repeated generic "we are all alike" text. Yet the DPO response opened with a stigmatizing "not black enough" statement and repeated a hashtag many times. The nominal win

should not be read as a successful safe response: both outputs were poor, and the DPO text retained discriminatory framing and severe repetition.

6.4 HH-RLHF DPO loss: humiliating a transgender coworker

Prompt: a request to humiliate a transgender coworker. Both models refused to help with humiliation and suggested respectful alternatives. The judge preferred SFT because its suggestions were more comprehensive. This is a useful counterexample: DPO retained a safe refusal, but did not improve response quality in this pair.

7. Limitations and reproducibility

The experiments used a single random seed (42), and the pairwise evaluation relied on one judge model, Google Gemini 3 Flash Preview, whose judgments may contain bias. The persona evaluation included only four prompts, so its 50%-win rate provides limited evidence. For the manual SFT and DPO runs, only epoch-end metrics were recorded; therefore, their plots do not show full training trajectories. Reproduction details are stored in the run-configuration and generation-manifest JSON files, while generated responses, pairwise comparisons, judgments, and summaries are saved under outputs/. Large model checkpoints are retained separately because of their size.

Reproduction artifacts are in the repository: outputs/*/run_config.json, outputs/*/metrics.jsonl, outputs/plots/, outputs/eval/, and outputs/evaluation/. The public adapter tests passed locally (7 passed). The recorded generation and judge settings are documented in the corresponding manifests and configuration files.

8. Conclusion

The pipeline demonstrates that manual instruction tuning can substantially improve a small base model on the tested prompts, while later preference optimization is not guaranteed to improve outputs judged externally. SFT beat the base model decisively in the Alpaca-style pairwise comparison. UltraFeedback DPO was approximately tied with SFT despite stronger recorded validation reward diagnostics. HH-RLHF DPO underperformed SFT in the saved pairwise set, and qualitative examples show that repetition, factual errors, and imperfect safety behavior remained. The tiny persona comparison is inconclusive. The primary lesson is to evaluate alignment changes using both training diagnostics and carefully inspected outputs, and to report regressions as well as gains.

References

[1] Assignment PA2 manual. “Assignment: Instruction Tuning and Direct Preference Optimization on a Small Language Model.” Course assignment handout, 24 pages, 2026.

[2] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. “Direct Preference Optimization: Your Language Model is Secretly a Reward Model.” Advances in Neural Information Processing Systems, 36, 2023. arXiv:2305.18290. https://arxiv.org/abs/2305.18290

[3] L. Ouyang et al. “Training language models to follow instructions with human feedback.” Advances in Neural Information Processing Systems, 35, 2022.

[4] Y. Dubois, B. Galambosi, P. Liang, and T. B. Hashimoto. “Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators.” COLM, 2024.

[5] Qwen Team. “Qwen2.5-0.5B model card.” Hugging Face. https://huggingface.co/Qwen/Qwen2.5-0.5B

[6] Hugging Face SmolLM Team. “SmolLM2-360M-Instruct model card.” Hugging Face. https://huggingface.co/HuggingFaceTB/SmolLM2-360M-Instruct

[7] Y. Bai et al. “Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback.” 2022. arXiv:2204.05862.